

СТАРТ

ДЕНЬ 30

Профессия, портфолио и следующий шаг

Собрать знания курса в портфолио-проект, описать свой стек и составить реалистичный план развития после 30 дней.

DAY 30

STACK

junior

middle

soft skills

portfolio

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Собрать знания курса в портфолио-проект, описать свой стек и составить реалистичный план развития после 30 дней.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Деплой, Docker, облако и CI/CD

Что уже есть в нашей модели.



СЕГОДНЯ

Профессия, портфолио и следующий шаг

Вчера: Деплой, Docker, облако и CI/CD. Сегодня мы добавляем следующий слой: профессия, портфолио и следующий шаг.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

После курса важнее не «знать всё», а уметь показать законченный проект, объяснить решения и системно закрывать пробелы.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Курс дал карту и базовые инструменты. Портфолио-проект — первый построенный дом: не идеальный небоскрёб, а доказательство, что ты умеешь пройти путь от идеи до результата.

ПЕРЕВОД В IT

stack

набор технологий, используемых в проекте

junior

начальный уровень специалиста, работающего с поддержкой

middle

уровень специалиста с большей самостоятельностью

Слой 1: stack • junior • middle

Сначала смысл, затем профессиональный термин.

**stack**

набор технологий, используемых в проекте

**junior**

начальный уровень специалиста, работающего с поддержкой

**middle**

уровень специалиста с большей самостоятельностью

Слой 2: soft skills • portfolio • interview

Сначала смысл, затем профессиональный термин.

**soft skills**

навыки общения, обратной связи и организации работы

**portfolio**

набор проектов, демонстрирующих практические навыки

**interview**

встреча для взаимной оценки кандидата и компании

Слой 3: resume • roadmap

Сначала смысл, затем профессиональный термин.

**resume**

краткое описание опыта и навыков кандидата

**roadmap**

последовательный план профессионального развития

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: mentor • feedback

Сначала смысл, затем профессиональный термин.

mentor

опытный специалист, помогающий в обучении

feedback

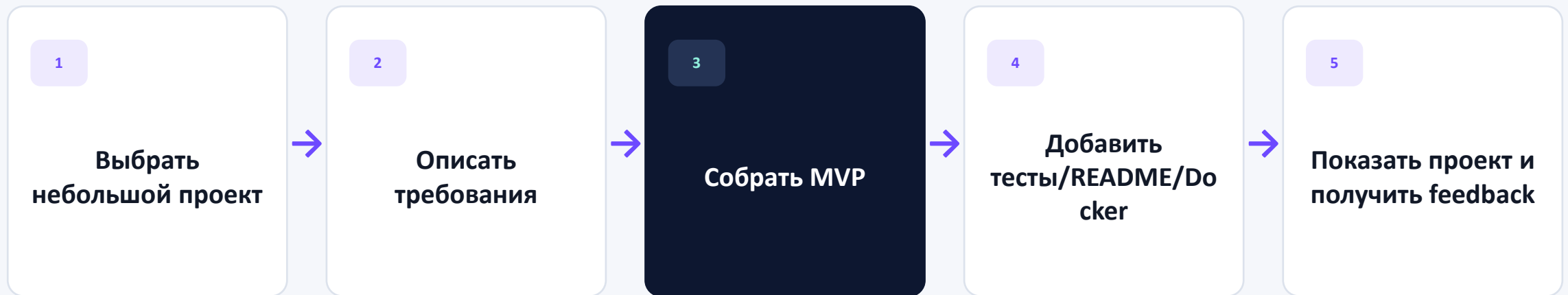
конкретная обратная связь о результате работы

Не заучивай формулировку. Придумай свой пример использования каждого термина.

СХЕМА

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.

```
def progress(done, total):  
    return done / total * 100  
  
print(progress(24, 30))
```

day30_step1

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
def progress(done, total):
```

Создаём функцию и даём ей имя; код внутри выполнится при вызове.

```
    return done / total * 100
```

Python выполняет эту строку сверху вниз.

```
print(progress(24, 30))
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
def progress(done, total):  
    return done / total * 100  
  
print(progress(24, 30))
```

РЕЗУЛЬТАТ

80.0

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
def progress(done, total):  
    if total == 0:  
        return 0  
    return done / total * 100  
  
print(progress(0, 0))
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

0

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
def progress(done, total):  
    return done / total * 100  
  
print(progress(0, 0))
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

При `total=0` происходит `ZeroDivisionError`. Даже простая функция должна учитывать разумный крайний случай.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

Исправленный вариант

Меняем только то, что действительно было причиной.

```
def progress(done, total):  
    if total == 0:  
        return 0  
    return done / total * 100
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Спроектировать итоговый проект «Трекер обучения» и разложить его на маленькие части.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Flask-приложение и понятные routes.**
- 2 База пользователей/занятий.
- 3 Авторизация и CRUD записей.
- 4 REST endpoint прогресса.
- 5 Тесты ключевой логики, Dockerfile и README с запуском.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
solution_part1

MVP = [  
    "Flask routes",  
    "database",  
    "auth",  
    "CRUD study sessions",  
    "REST progress endpoint",  
    "pytest",
```

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.

```
MVP = [  
    "Flask routes",  
    "database",  
    "auth",  
    "CRUD study sessions",  
    "REST progress endpoint",  
    "pytest",  
    "Dockerfile",  
    "README"  
]  
for item in MVP:  
    print("-", item)
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

Чёткий список частей итогового проекта вместо одной огромной задачи

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Подготовь 60-секундный рассказ о проекте: проблема → что сделал → стек → самое сложное → чему научился.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Не перечисляй технологии без контекста. Связывай каждую с задачей, которую она решала.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
pitch = ["Проблема", "Решение", "Стек", "Сложность", "Результат"]  
for part in pitch:  
    print(part)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Структура короткой презентации проекта

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Составь план на 8 недель: улучшение проекта, алгоритмы, SQL/API, тесты, Git, резюме и первые отклики.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
weeks = {  
    1: "полировка MVP", 2: "SQL и данные",  
    3: "API", 4: "тесты",  
    5: "Docker/deploy", 6: "Git + review",  
    7: "README + CV", 8: "отклики + feedback"  
}  
for week, focus in weeks.items():  
    print(week, focus)
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Восемь недель с конкретным фокусом вместо расплывчатого «учить дальше»

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Пытаться изучать всё сразу.

02

Делать портфолио-проект слишком большим.

03

Копировать код, который не можешь объяснить.

04

Ждать идеальной готовности до первой обратной связи.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «stack» и какую проблему это понятие помогает описать?

2

Чем «stack» отличается от «soft skills»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **stack — набор технологий, используемых в проекте.**

2 stack: набор технологий, используемых в проекте. soft skills: навыки общения, обратной связи и организации работы.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 **80.0**

4 При `total=0` происходит `ZeroDivisionError`. Даже простая функция должна учитывать разумный крайний случай.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ **Собрать знания курса в портфолио-проект, описать свой стек и составить реалистичный план развития после 30 дней.**
- ✓ Читать и объяснять пример темы «Профессия, портфолио и следующий шаг».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Курс завершён

Связываем сегодняшний результат со следующим шагом.

Теперь главное — повторение и собственный проект.

Курс завершён. Дальше — закрепление через проект, повторение и реальную практику.

Ты прошёл фундамент. Не пытайся знать всё — закрепляй то, что можешь объяснить и применить.